

Week-5-S-2520080033

Session-9

Task-1

To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>
#include <errno.h>

#define MAX_INPUT 100

char previous_dir[PATH_MAX] = "";

/* Change directory command */
void change_directory(char *path)
{
    char current_dir[PATH_MAX];

    if (getcwd(current_dir, sizeof(current_dir)) == NULL) {
        perror("getcwd");
        return;
    }

    /* If no path is given, go to HOME */
    if (path == NULL || strlen(path) == 0) {
        path = getenv("HOME");
    }

    if (path == NULL) {
        printf("HOME directory not found.\n");
        return;
    }

    /* Previous directory */
    if (strcmp(path, "-") == 0) {
        if (strlen(previous_dir) == 0) {
            printf("No previous directory.\n");
            return;
        }
        path = previous_dir;
    }

    if (chdir(path) != 0) {
        perror("cd");
        return;
    }

    /* Save previous directory */
    strcpy(previous_dir, current_dir);

    printf("Current directory: %s\n", getcwd(current_dir, sizeof(current_dir)));
}

/* Print current working directory */
void print_working_directory()
{
    char cwd[PATH_MAX];

    if (getcwd(cwd, sizeof(cwd)) != NULL)
        printf("%s\n", cwd);
    else
        perror("getcwd");
}

/* Command function type */
typedef void (*command_function)(char *);

/* Built-in command table */
struct command {
    char *name;
    command_function function;
};

/* Built-in commands */
void pwd_command(char *arg)
{
    print_working_directory();
}

void cd_command(char *arg)
{
    change_directory(arg);
}
```

```

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strlen(input) == 0)
            continue;

        command = strtok(input, " ");
        argument = strtok(NULL, "");

        /* Exit command */
        if (strcmp(command, "exit") == 0) {
            printf("Exiting shell...\n");
            break;
        }

        int found = 0;

        /* Dispatch command */
        for (int i = 0; i < command_count; i++) {

            if (strcmp(command, commands[i].name) == 0) {

                commands[i].function(argument);
                found = 1;
                break;
            }
        }

        /* Invalid command */
        if (!found) {
            printf("Invalid command: %s\n", command);
            printf("Type 'help' for available commands.\n");
        }
    }

    return 0;
}

```

This message is shown once a day. To disable it please create the /root/.hushlogin file.

```

root@Shamikahasini:~# nano osskill3.c
root@Shamikahasini:~# gcc osskill3.c -o osskill3
root@Shamikahasini:~#
root@Shamikahasini:~# ./osskill3
Current Directory: /root

```

```

Enter directory path (or 'prev' / 'exit'): /tmp
Directory changed successfully.
Current Directory: /tmp

```

```

Enter directory path (or 'prev' / 'exit'): /home
Directory changed successfully.
Current Directory: /home

```

```

Enter directory path (or 'prev' / 'exit'): prev
Changed to previous directory: /tmp

```

```

Enter directory path (or 'prev' / 'exit'): abc
No such file or directory
Invalid path:
Enter directory path (or 'prev' / 'exit'): exit
root@Shamikahasini:~# |

```

Observation:

The program successfully changes the current working directory using `chdir()`, validates the given path, updates and displays the working directory using `getcwd()`, handles invalid paths using `perror()`, and maintains the previous directory for navigation back to it.

Task-2

TO Identify Built-ins, Create Dispatch Table, Execute In- Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

void execute_cd(char *arg) {
    if (chdir(arg) != 0)
        perror("cd");
}

void execute_pwd() {
    char cwd[1024];

    if (getcwd(cwd, sizeof(cwd)) != NULL)
        printf("%s\n", cwd);
    else
        perror("pwd");
}

void execute_exit() {
    printf("Exiting shell...\n");
    exit(0);
}

int main() {
    char command[100];
    char *arg;

    while (1) {
        printf("myshell> ");
        scanf("%s", command);

        if (strcmp(command, "cd") == 0) {
            scanf("%s", arg = malloc(100));
            execute_cd(arg);
            free(arg);
        }
        else if (strcmp(command, "pwd") == 0) {
            execute_pwd();
        }
        else if (strcmp(command, "exit") == 0) {
            execute_exit();
        }
        else if (strcmp(command, "ls") == 0 ||
                 strcmp(command, "date") == 0 ||
                 strcmp(command, "whoami") == 0) {

            pid_t pid = fork();

            if (pid == 0) {
                execlp(command, command, NULL);
                perror("Command execution failed");
                exit(1);
            }
            else {
                wait(NULL);
            }
        }
        else {
            printf("Invalid command: %s\n", command);
        }
    }

    return 0;
}
```

```

root@Shamikahasini:~# nano osskill4.c
root@Shamikahasini:~# gcc osskill4.c -o osskill4
root@Shamikahasini:~# ./osskill4
myshell> pwd
/root
myshell> ls
FIRIST.c      command_resolver.c  fork_demo.c  osskill4      process      sample.txt  task1.c  task4s.c  waitpid_demo.c
FIRIST.c.save destination.txt      memory.c     osskill4.c    process.c    signal_project task2.c  trace.txt  waitpid_demo.c
OSSP          fifo_project        memory.c     parser        process_state source.txt  task2.c  wait_example week-2
OSSP-2        filecopy            memory_lab   parser.c      process_state state_test  task3.c  wait_example.c week2
OSSP1         filecopy.c          osskill3     pipe_exec     producer_consumer state_test.c task3.c  wait_waitpid zombie
command_resolver fork_demo           osskill3.c   pipe_exec.c   producer_consumer task1      task4s  wait_waitpid.c zombie.c
myshell> date
Thu Sep 17 04:51:48 UTC 2026
myshell> whoami
root
myshell> cd /temp
cd: No such file or directory
myshell> pwd
/root
myshell> abc
Invalid command: abc
myshell> exit
Exiting shell...
root@Shamikahasini:~# |

```

Observation

The program identifies built-in commands such as `cd`, `pwd`, and `exit` using a dispatch logic. Non-built-in commands such as `ls`, `date`, and `whoami` are executed using `fork()` and `execlp()`. Invalid commands are detected and reported, while the shell maintains its working-directory state across commands.

Session-10

Task-1

To Retrieve Current Directory, Display path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <limits.h>
#include <string.h>

int main() {
    char cwd[PATH_MAX];
    char saved_state[PATH_MAX];
    char command[20];

    /* Retrieve current directory */
    if (getcwd(cwd, sizeof(cwd)) == NULL) {
        perror("getcwd");
        return 1;
    }

    /* Save current state */
    strcpy(saved_state, cwd);

    printf("Current Directory: %s\n", cwd);
    printf("State saved successfully.\n");

    while (1) {
        printf("\nEnter command (pwd/exit): ");
        scanf("%s", command);

        /* Display current path */
        if (strcmp(command, "pwd") == 0) {
            if (getcwd(cwd, sizeof(cwd)) != NULL)
                printf("Current Path: %s\n", cwd);
            else
                perror("getcwd");
        }

        /* Process exit request */
        else if (strcmp(command, "exit") == 0) {
            printf("Exit request received.\n");
            printf("Saved State: %s\n", saved_state);
        }
    }
}

```

```

        printf("Resources cleaned up.\n");
        printf("Program terminated successfully.\n");
        break;
    }

    /* Handle invalid commands */
    else {
        printf("Invalid command.\n");
    }
}

return 0;
}

```

```

root@Shamikahasini:~# gcc program1.c -o program
root@Shamikahasini:~# ./program
Current Directory: /root
State saved successfully.

Enter command (pwd/exit): pwd
Current Path: /root

Enter command (pwd/exit): abc
Invalid command.

Enter command (pwd/exit): exit
Exit request received.
Saved State: /root
Resources cleaned up.
Program terminated successfully.
root@Shamikahasini:~# |

```

Observation

The program successfully retrieves and displays the current working directory, saves the directory state, processes pwd and exit commands, handles invalid commands, and terminates correctly after performing the required cleanup.

Task-2

To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <unistd.h>
#include <sys/wait.h>

int valid_name(char *name) {
    if (!isalpha(name[0]) && name[0] != '_')
        return 0;

    for (int i = 1; name[i] != '\0'; i++) {
        if (!isalnum(name[i]) && name[i] != '_')
            return 0;
    }

    return 1;
}

int main() {
    char input[200];
    char *name, *value;
    char *equal;

    while (1) {
        printf("myshell> ");
        fgets(input, sizeof(input), stdin);

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        if (strncmp(input, "export ", 7) == 0) {
            char *assignment = input + 7;

            equal = strchr(assignment, '=');

            if (equal == NULL) {
                printf("Invalid export syntax.\n");
                continue;
            }

            *equal = '\0';
            name = assignment;
            value = equal + 1;

            if (!valid_name(name)) {
                printf("Invalid variable name.\n");
                continue;
            }

            if (setenv(name, value, 1) == 0)
                printf("Variable exported successfully.\n");
            else
                perror("setenv");
        }

        else if (strcmp(input, "test") == 0) {
            pid_t pid = fork();

            if (pid == 0) {
                char *args[] = {"printenv", "MYVAR", NULL};
                execvp("printenv", args);
                perror("execvp");
                exit(1);
            }
            else {
                wait(NULL);
            }
        }

        else {
            printf("Invalid command.\n");
        }
    }

    return 0;
}
```

```
root@Shamikahasini:~# nano program2.c
root@Shamikahasini:~# gcc program2.c -o program2
root@Shamikahasini:~# ./program2
myshell> export MYVAR=Hello
Variable exported successfully.
myshell> test
Hello
myshell> export MYVAR=Welcome
Variable exported successfully.
myshell> test
Welcome
myshell> export 123VAR=abc
Invalid variable name.
myshell> export MYVAR
Invalid export syntax.
myshell> exit
root@Shamikahasini:~# |
```

Observation

The program successfully parses the export command, validates environment variable names, creates and updates existing variables, passes exported variables to child processes, handles invalid syntax and names, and verifies the functionality successfully.